

Group 3



DIGITAL MARKETING

Predicting Customer Conversion

MAO Kolbotr Pissoth
LACH Sovitou



CONTENT



1 DATASET SELECTION

2 MODEL BUILDING

EDA

Selection

Training

Evaluation

Improvement

3 RESULT

3 REAL WORLD IMPACT

OBJECTIVE

The Challenge

In e-commerce, marketing teams face a constant "tug-of-war" between two goals: Growth (preventing losing customers to competitors) and Efficiency (preventing budget waste on marketing).

The Solution

Using a dataset of 8,000 customers, we aim to develop a model designed for this Balanced Intelligence.

The Success Metric

By prioritizing a high Macro-Average F1, we ensured the model respects the logic of both class (converted and non-converted) behaviors equally.

This provides the business with a reliable engine that captures revenue without blindly wasting resources.



DATASET SELECTION

Name: Predict Conversion in Digital Marketing Dataset

Description

A synthetic dataset designed to simulate realistic digital marketing data from an e-commerce environment made by Mr. Rabie El Kharoua

Key Characteristics

- Size: 8,000 rows × 20 columns
- Target Variable: Conversion (1 = Customer converted, 0 = Did not convert)
- Feature Types:
 - Numerical (Age, Income, AdSpend, ClickThroughRate, TimeOnSite, etc.)
 - Categorical (Gender, CampaignChannel, CampaignType)
- Task Type: Binary Classification



DATASET SELECTION

Dataset Snapshot

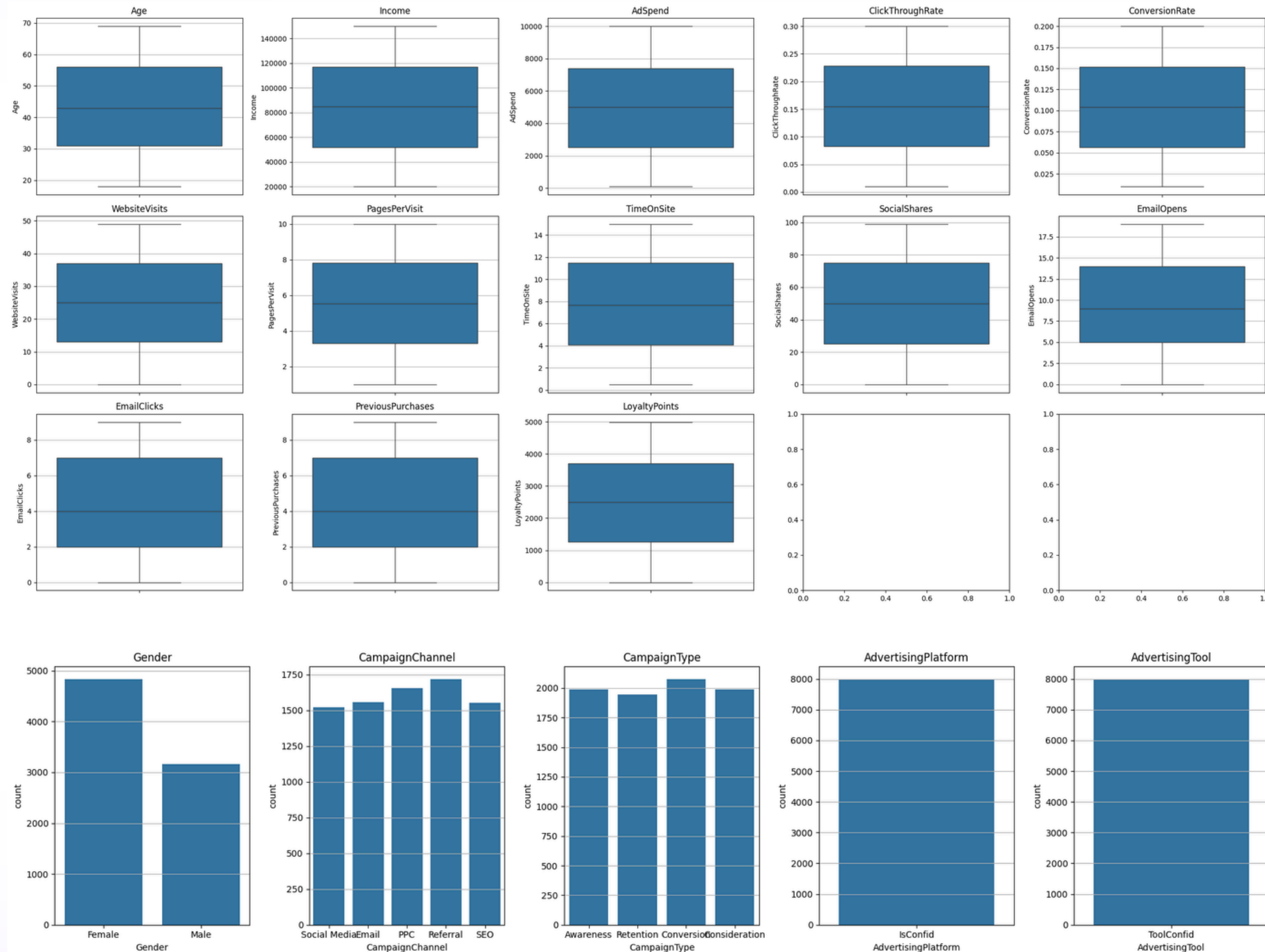
	CustomerID	Age	Gender	Income	CampaignChannel	CampaignType	AdSpend	ClickThroughRate	ConversionRate	WebsiteVisits
0	8000	56	Female	136912	Social Media	Awareness	6497.870068	0.043919	0.088031	0
1	8001	69	Male	41760	Email	Retention	3898.668606	0.155725	0.182725	42
2	8002	46	Female	88456	PPC	Awareness	1546.429596	0.277490	0.076423	2
3	8003	32	Female	44085	PPC	Conversion	539.525936	0.137611	0.088004	47
4	8004	60	Female	83964	PPC	Conversion	1678.043573	0.252851	0.109940	0

PagesPerVisit	TimeOnSite	SocialShares	EmailOpens	EmailClicks	PreviousPurchases	LoyaltyPoints	AdvertisingPlatform	AdvertisingTool	Conversion
2.399017	7.396803	19	6	9	4	688	IsConfid	ToolConfid	1
2.917138	5.352549	5	2	7	2	3459	IsConfid	ToolConfid	1
8.223619	13.794901	0	11	2	8	2337	IsConfid	ToolConfid	1
4.540939	14.688363	89	2	2	0	2463	IsConfid	ToolConfid	1
2.046847	13.993370	6	6	6	8	4345	IsConfid	ToolConfid	1

DATASET SELECTION

Data Cleaning

We've found no Duplicates, Missing Values or Outliers



```
df.duplicated().sum()
```

✓ 0.0s

```
np.int64(0)
```

```
df.isna().sum()
```

✓ 0.0s

CustomerID	0
Age	0
Gender	0
Income	0
CampaignChannel	0
CampaignType	0
AdSpend	0
ClickThroughRate	0
ConversionRate	0
WebsiteVisits	0
PagesPerVisit	0
TimeOnSite	0
SocialShares	0
EmailOpens	0
EmailClicks	0
PreviousPurchases	0
LoyaltyPoints	0
AdvertisingPlatform	0
AdvertisingTool	0
Conversion	0

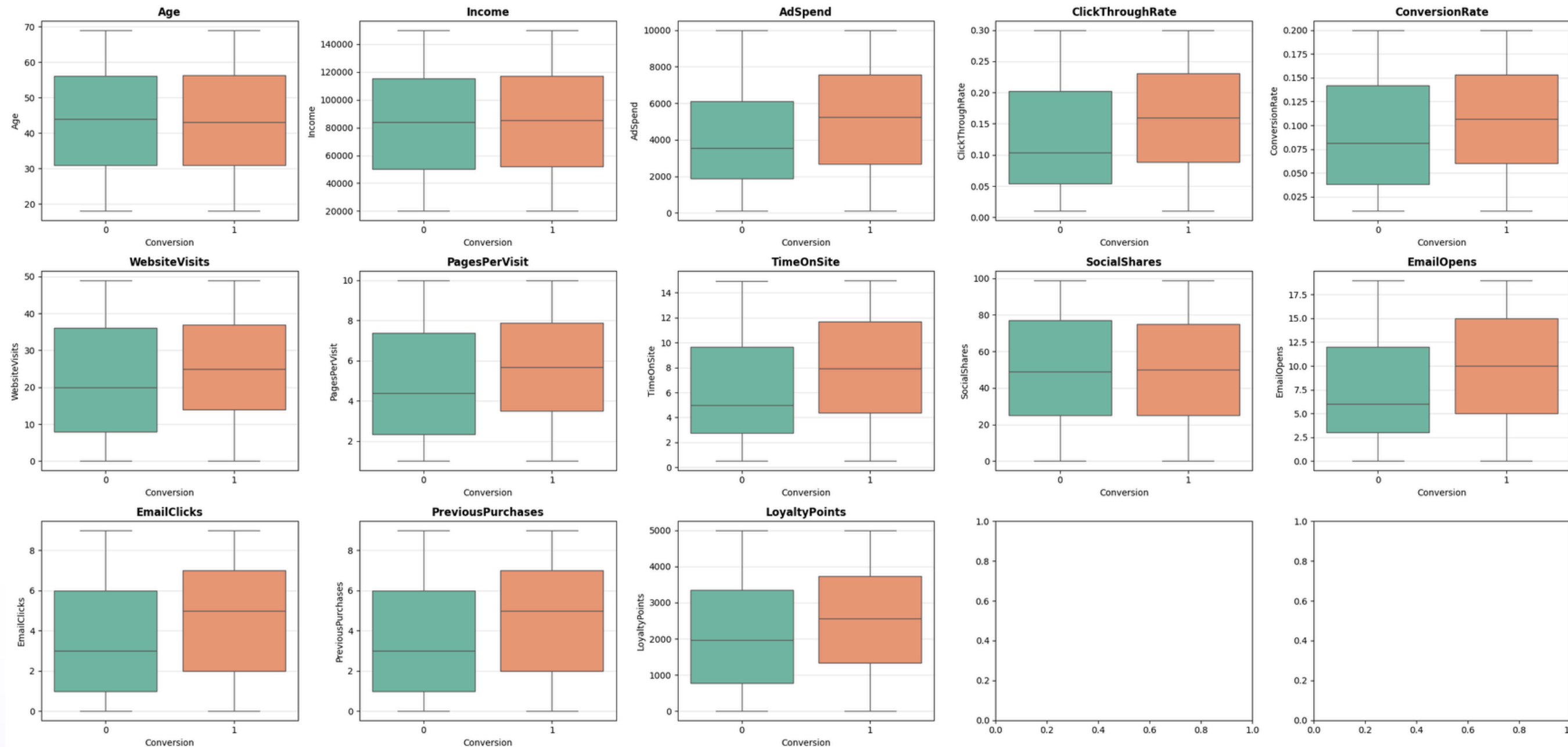
dtype: int64

DATASET SELECTION

Data Exploration

Visualize Potential Relationship Between Conversion Column and Other Columns

Numerical Data

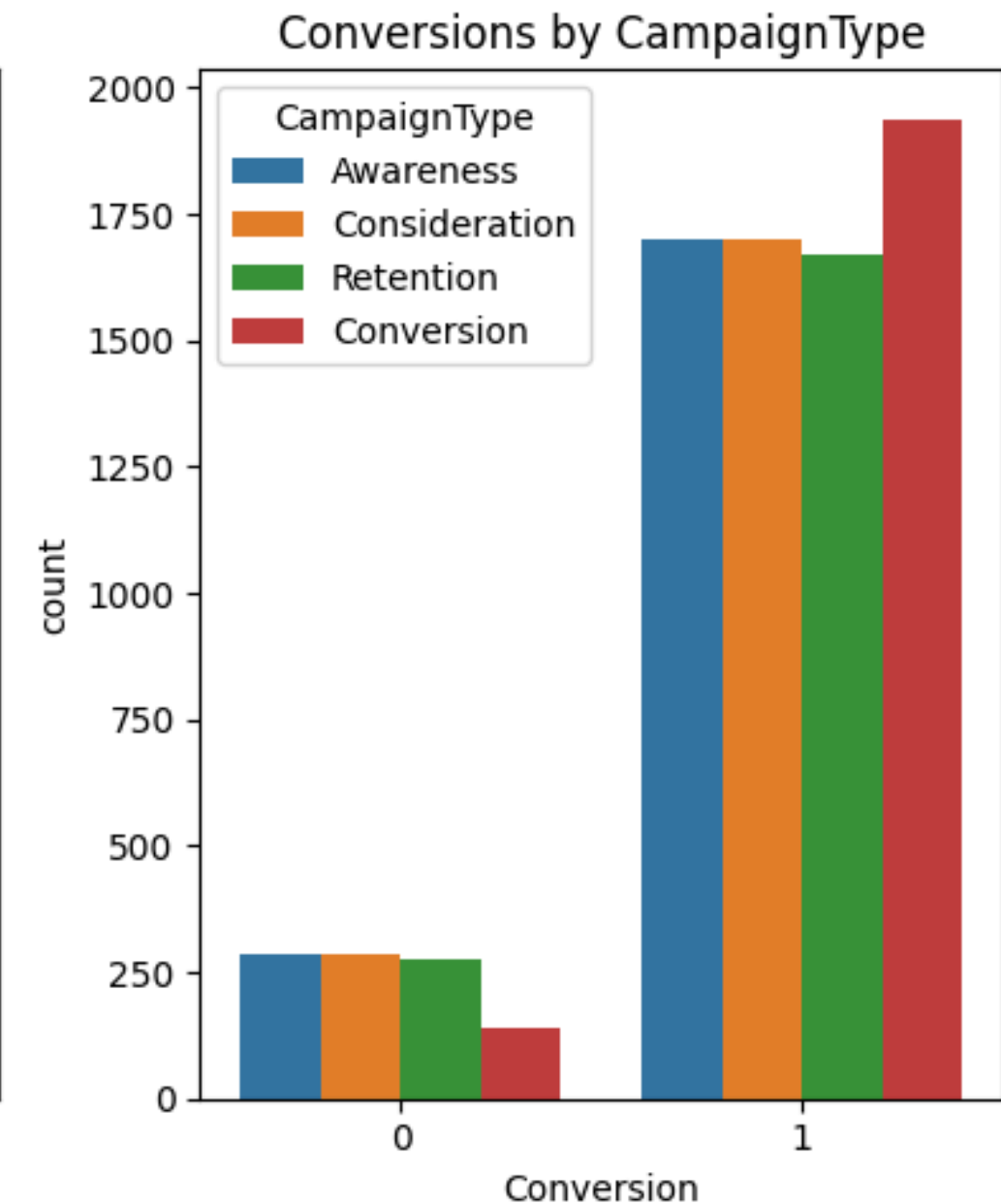
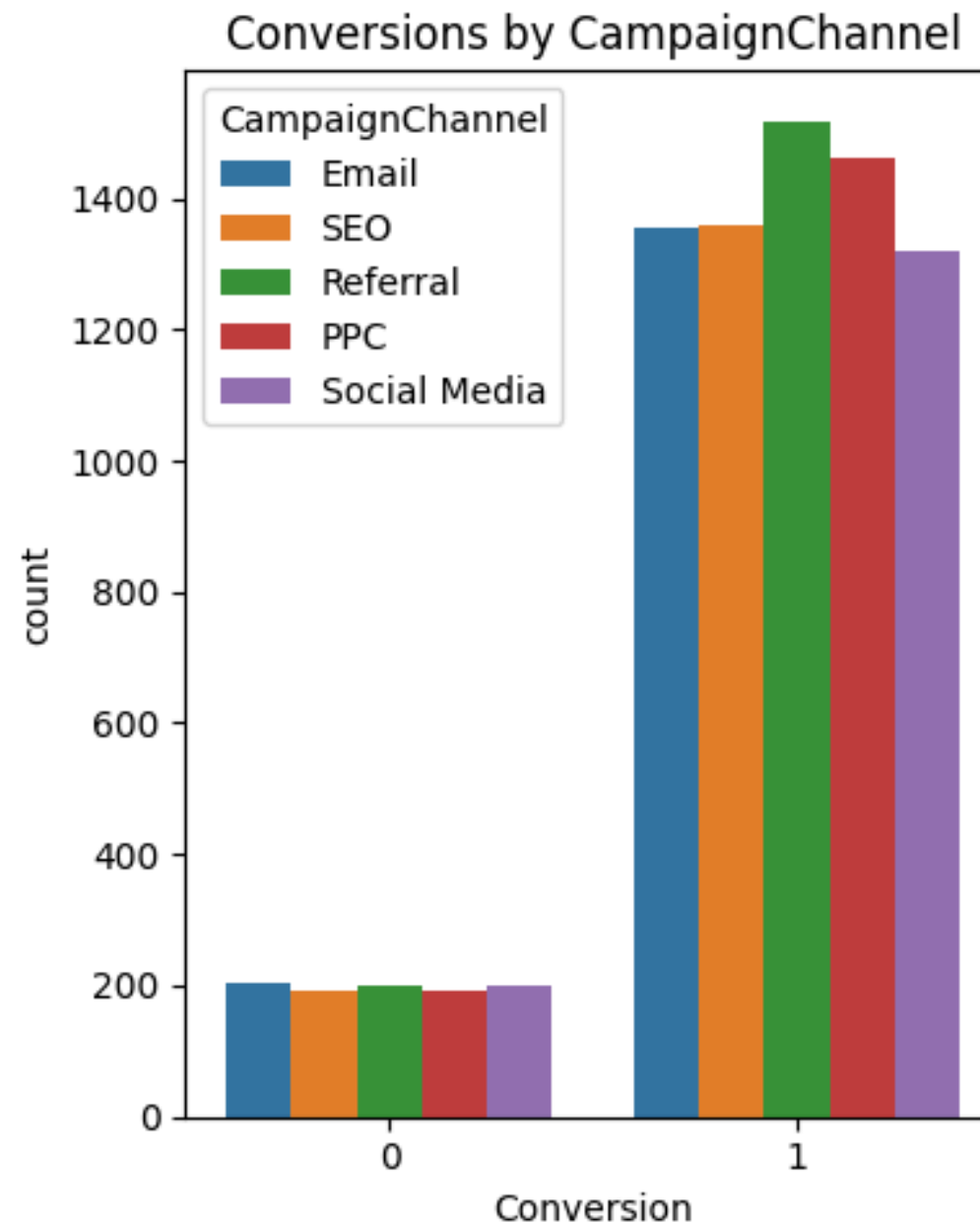
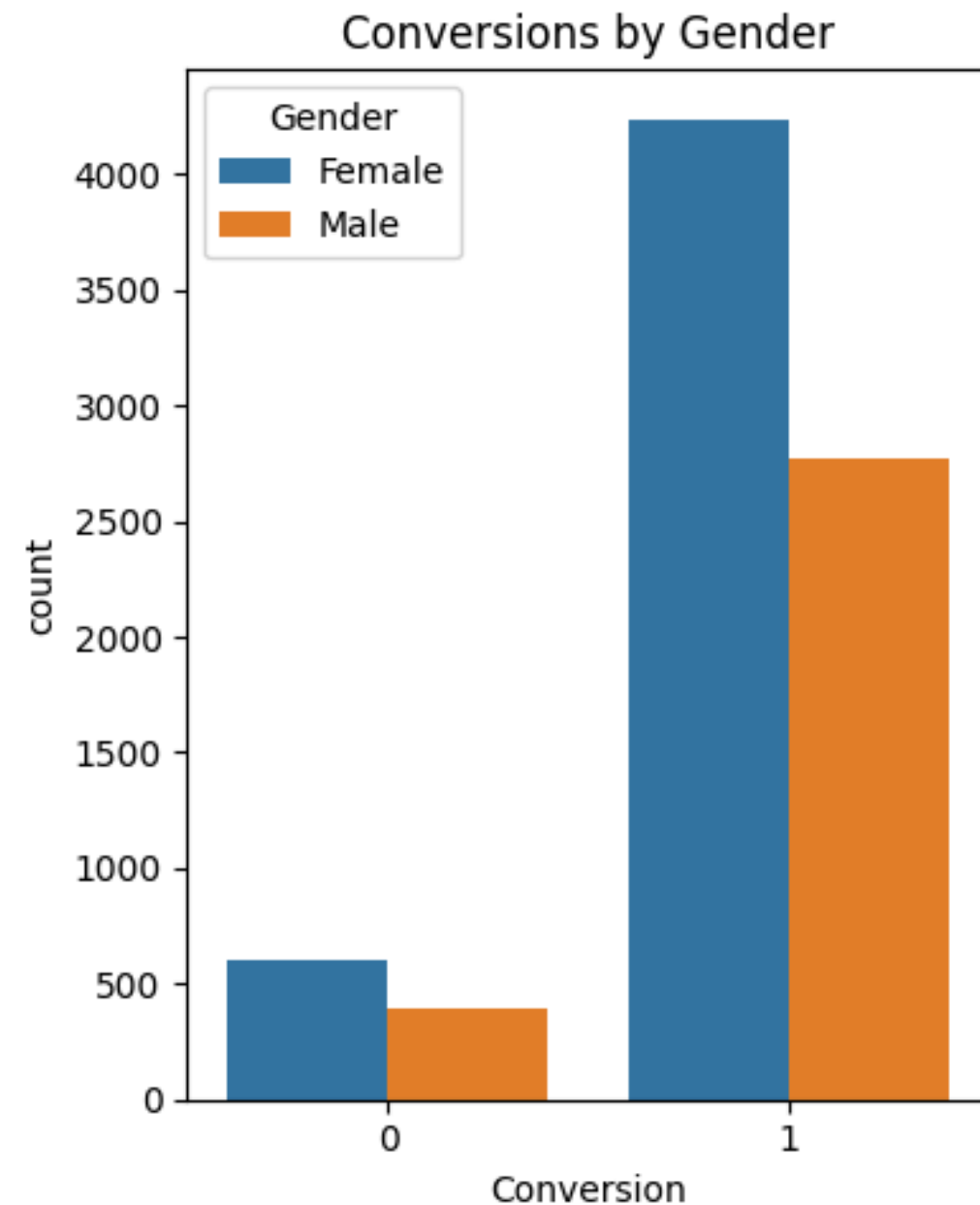


DATASET SELECTION

Data Exploration

Visualize Potential Relationship Between Conversion Column and Other Columns

Categorical Data



MODEL BUILDING

```
X_train, X_test, y_train, y_test =  
train_test_split(X, y, test_size=0.2,  
random_state=42, stratify=y)
```

Data Preprocessing + Splitting

One Hot Encode Categorical Columns with `pd.get_dummies()`

CampaignChannel	CampaignChannel_PPC	CampaignChannel_Referral	CampaignChannel_SEO	CampaignChannel_Social Media
Social Media	0	0	0	1
Email	0	0	0	0
PPC	1	0	0	0
PPC	1	0	0	0
PPC	1	0	0	0

Scale Numerical Columns with `StandardScaler()`

Age	Income	AdSpend	ClickThroughRate
56	136912	6497.870068	0.043919
69	41760	3898.668606	0.155725
46	88456	1546.429596	0.277490
32	44085	539.525936	0.137611
60	83964	1678.043573	0.252851

Age	Income	AdSpend	ClickThroughRate
0.830400	1.390381	0.527484	-1.320328
1.702775	-1.141736	-0.388418	0.010671
0.159343	0.100905	-1.217296	1.460225
-0.780138	-1.079865	-1.572106	-0.204964
1.098823	-0.018633	-1.170918	1.166907

MODEL BUILDING

Model Selection

As we're not familiar with the models that would fit this dataset, we decided to experiment with 8 common classification algorithms

- Linear/Simple: Logistic Regression, Gaussian Naive Bayes
- Tree-based: Decision Tree, Random Forest
- Boosting: Gradient Boosting, AdaBoost, XGBoost
- Instance-based: K-Nearest Neighbors
- Deep Learning: Sequential Neural Network (Sigmoid)

MODEL BUILDING

Model Selection

```
def model_functionality(model, x_train, y_train, x_test, y_test, model_name):  
    class_model = model  
    class_model.fit(x_train, y_train)  
  
    # Predict on the test set  
    predicted_y_test = class_model.predict(x_test)  
  
    return class_model
```

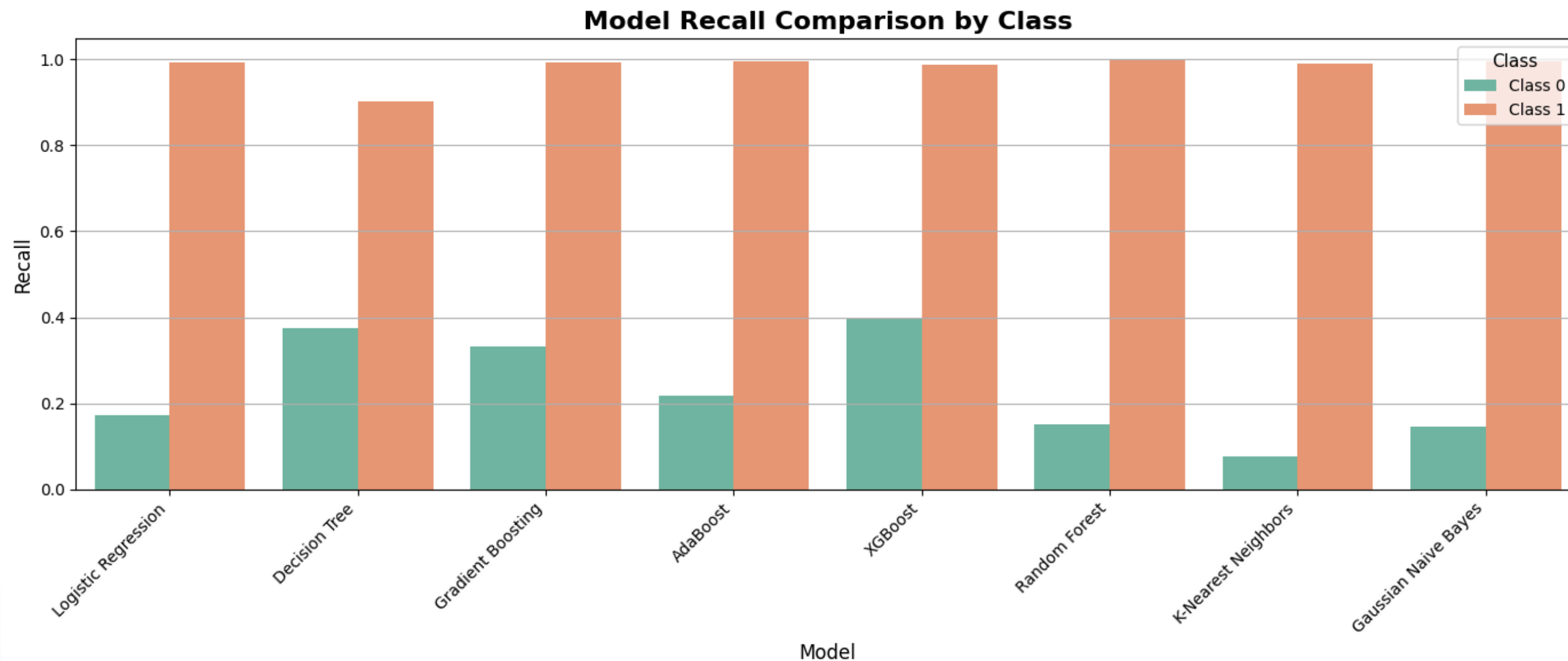
```
models = {  
    'Logistic Regression': LogisticRegression(),  
    'Decision Tree': DecisionTreeClassifier(),  
    'Gradient Boosting': GradientBoostingClassifier(),  
    'AdaBoost': AdaBoostClassifier(),  
    'XGBoost': XGBClassifier(),  
    'Random Forest': RandomForestClassifier(),  
    'K-Nearest Neighbors': KNeighborsClassifier(),  
    'Gaussian Naive Bayes': GaussianNB(),  
}  
  
# Train each model  
all_trained_models = {}  
for name, model in models.items():  
    trained_model = model_functionality(model, x_train, y_train, x_test, y_test, name)
```

MODEL BUILDING

Initial Performance

All models achieved 86-92% accuracy

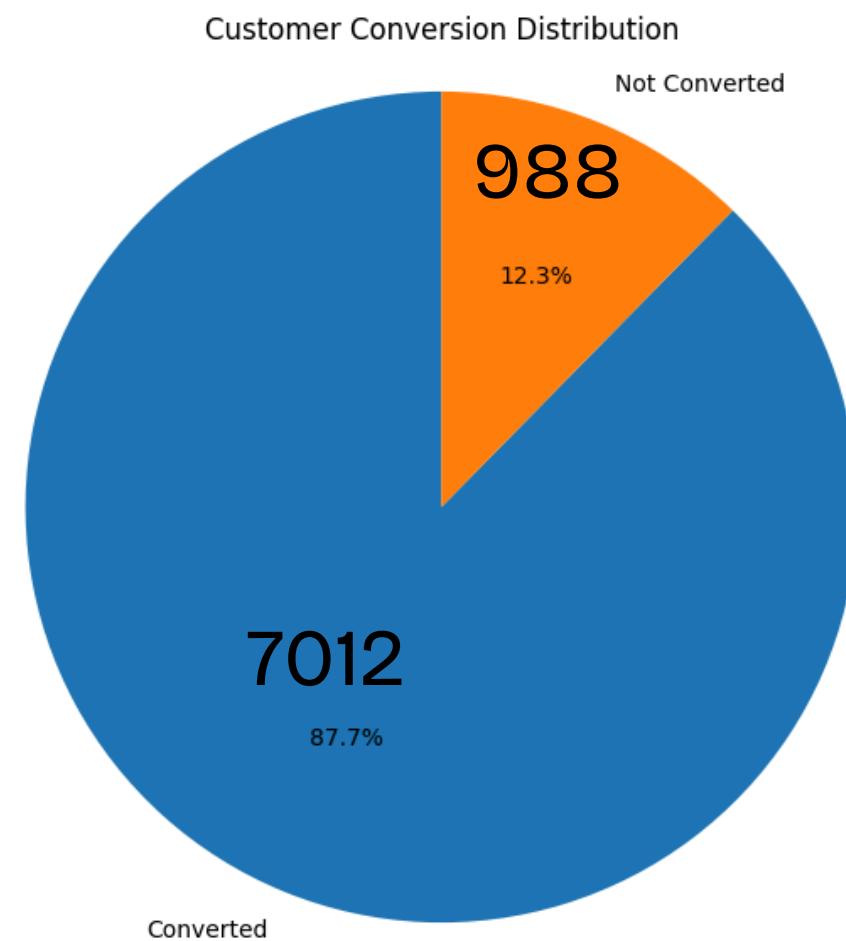
→ However, when we looked closer at the classification reports, we saw they were performing very poorly on Class 0 (non-converted customers)



MODEL BUILDING

Problem: Class Imbalance

We found out our target Conversion column has a very big class imbalance



This mean that even if the model is incapable and keep on guessing 1 [converted] most of the time, it will still score very high acuracy of 80%+ everytime.

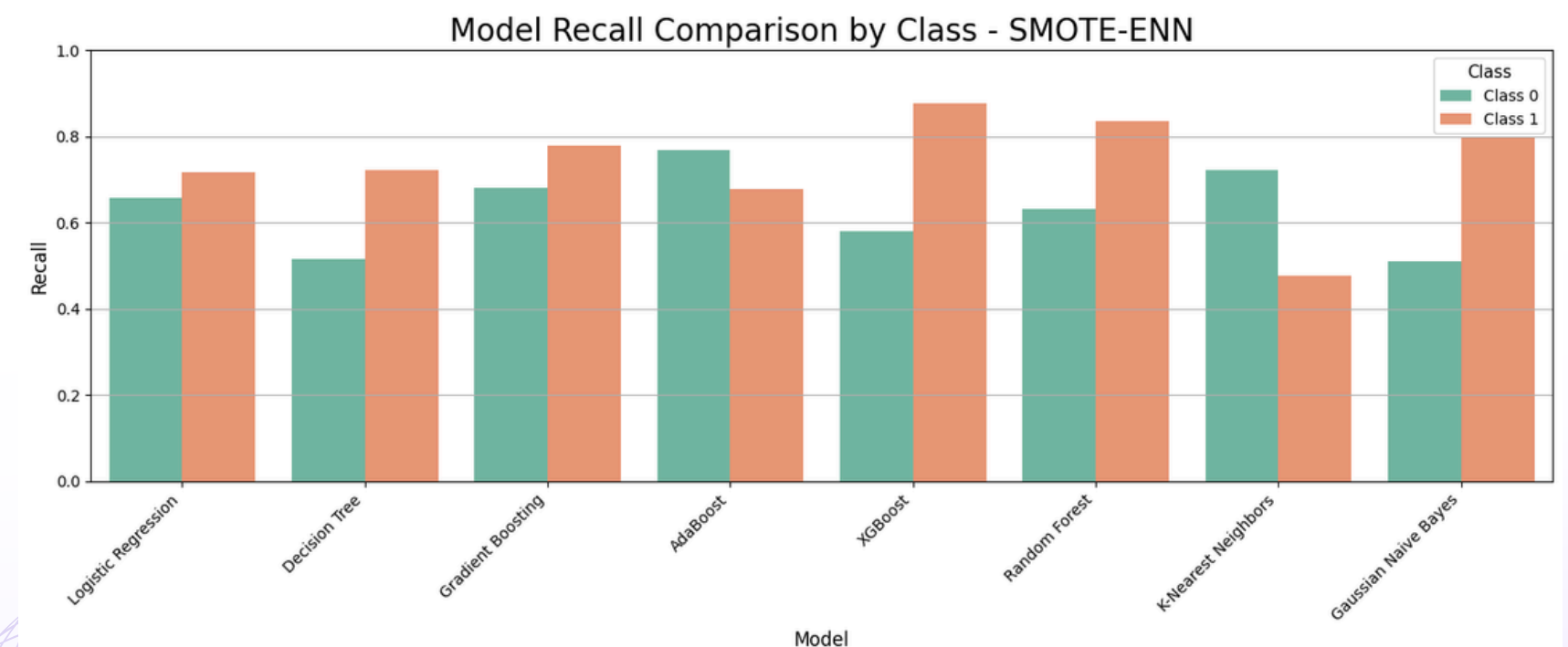
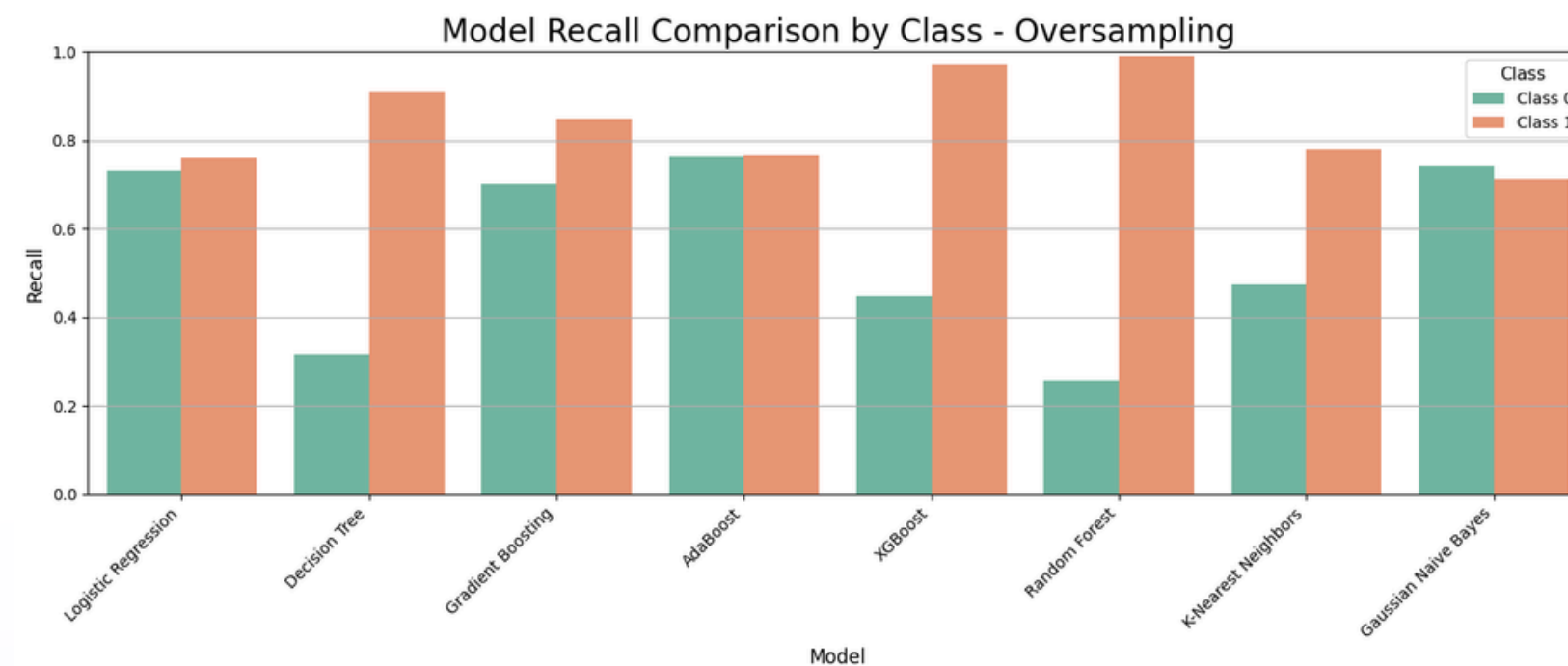
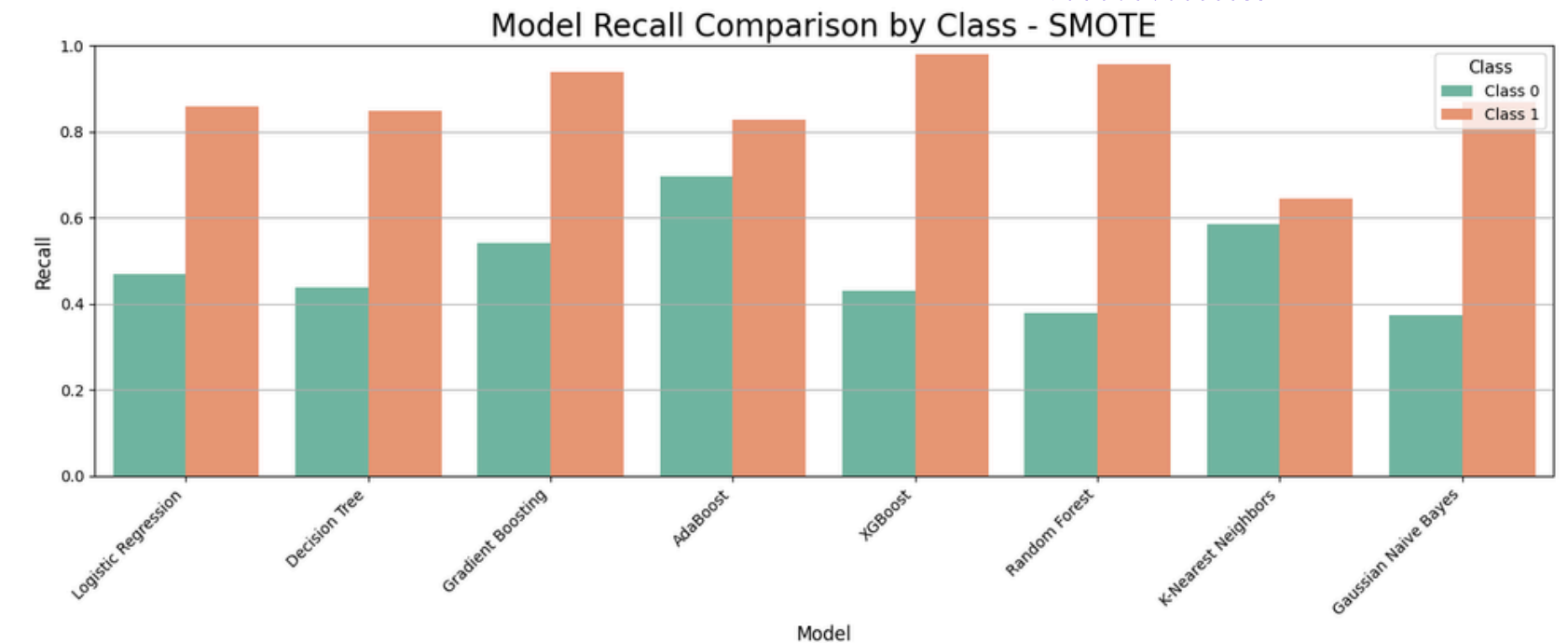
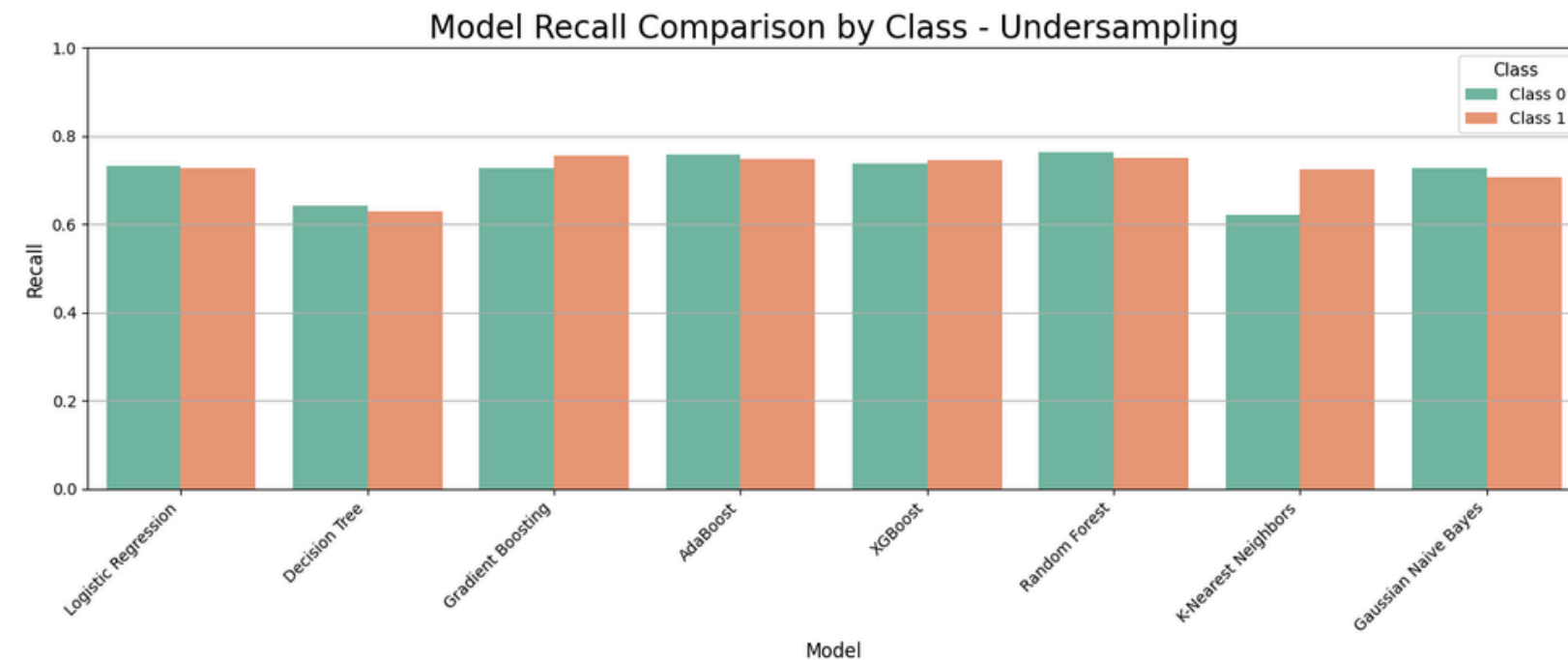
This is why we introduced techniques like

- resampling (undersampling, oversampling, SMOTE, SMOTEEN)
- class weighting to try and mitigate this bias and encourage the models to learn from and predict both classes more effectively.

MODEL BUILDING

Tackling Class Imbalance

We first tried different resampling techniques but while they improved Class 0, they hurt overall performance.

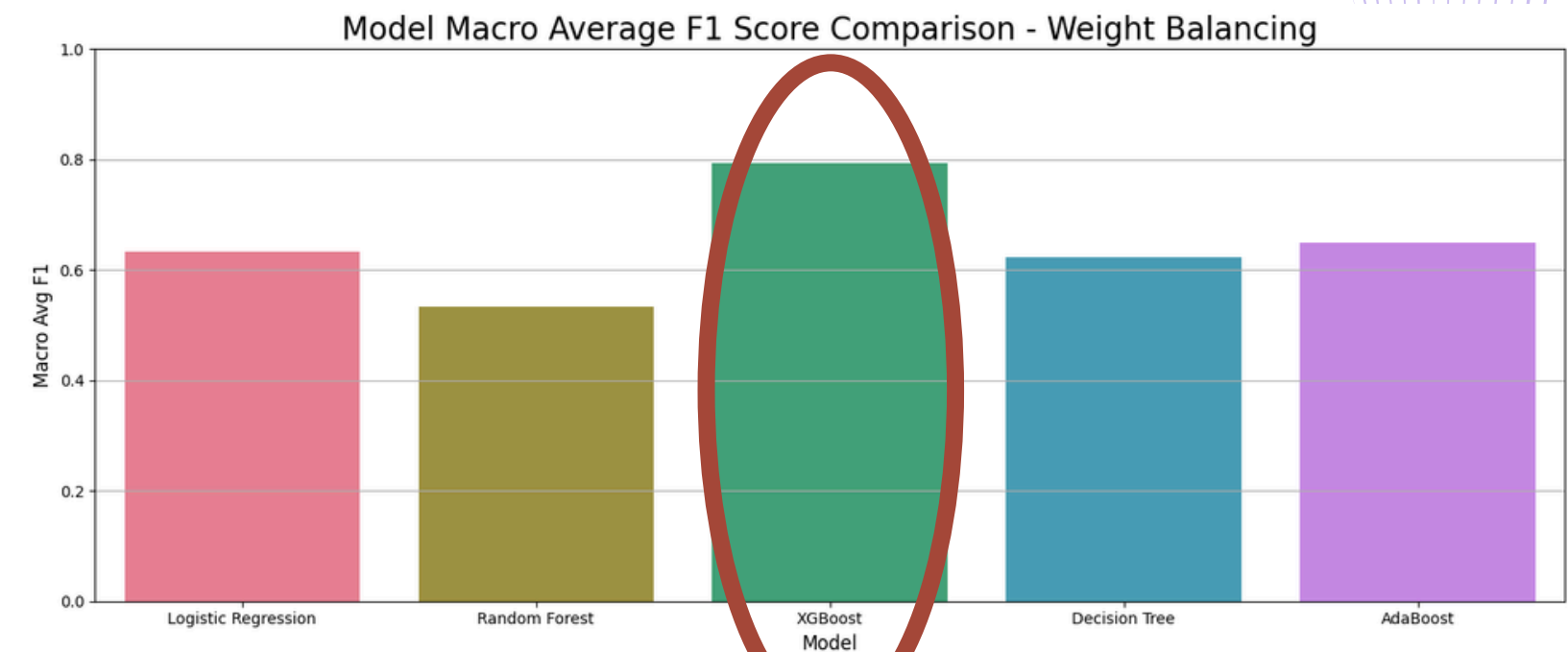
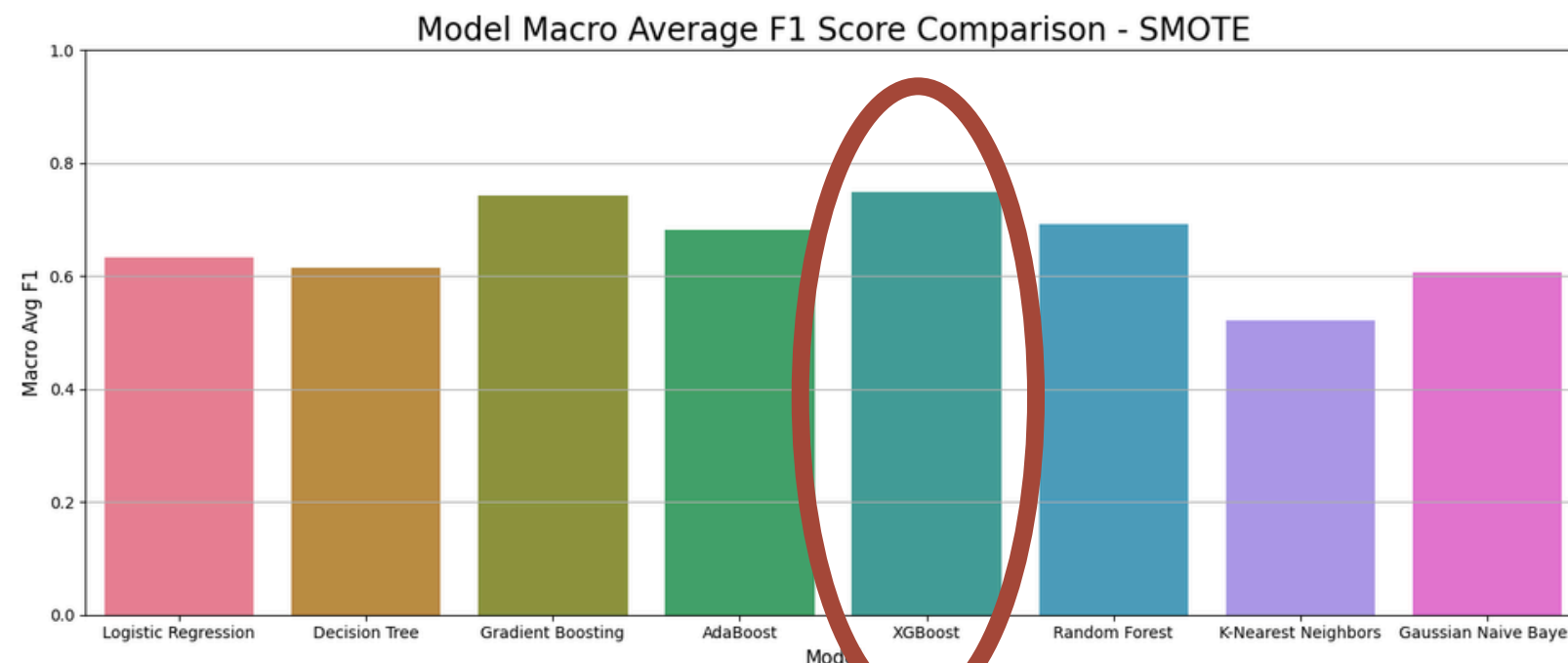


MODEL BUILDING

Tackling Class Imbalance

We then switched to class weighting and we found our best Macro Average F1 = 0.79

	precision	recall	f1-score	support
0	0.76	0.54	0.63	198
1	0.94	0.98	0.96	1402
accuracy			0.92	1600
macro avg	0.85	0.76	0.79	1600
weighted avg	0.92	0.92	0.92	1600



class weighting was implemented across various models to counter the observed class imbalance and improve the models' ability to accurately predict the minority class ('not converted').

It involves assigning different weights to the samples of each class. The minority class is given a higher weight, making its misclassifications more 'costly' to the model during training.

MODEL BUILDING

Selecting Best Classification Algorithm

Based on the results, XGBoost is the best performing model out of all:

- Ensemble Method: Builds upon a sequence of many "weak" decision trees.
- Gradient Boosting: Each new tree learns to correct the prediction errors (residuals) of the previous trees, iteratively improving the model's accuracy.
- Regularization: Incorporates built-in L1 and L2 regularization techniques to prevent overfitting, ensuring the model generalizes well to unseen data.
- Scalability: Designed for efficiency, it can handle large datasets and missing values

XG Boost works well due to:

- Effective Class Imbalance Handling: Crucially addressed the significant imbalance between 'Converted' and 'Not Converted' classes.
 - By utilizing `scale_pos_weight` in its configuration, XGBoost was directed to pay more attention to the minority class (0 - not converted).
 - This enabled the model to learn meaningful patterns from both classes, overcoming the bias observed in other models.

MODEL BUILDING

XGBoost Optimization

We selected XGBoost as our best base model and further improved it using **hyperparameter tuning** with RandomizedSearchCV.

```
param_grid = {
    'n_estimators': [200, 300, 400],
    'max_depth': [4, 6, 8, 10],
    'learning_rate': [0.05, 0.1, 0.15],
    'subsample': [0.7, 0.8, 0.9],
    'colsample_bytree': [0.7, 0.8, 0.9],
    'gamma': [0, 1, 2],
    'scale_pos_weight': [0.12, 0.15, 0.18]
}
xgb = XGBClassifier(random_state=42, eval_metric='aucpr')
search = RandomizedSearchCV(xgb, param_grid, n_iter=30, cv=5, scoring='f1_macro',
                             search.fit(X_train, y_train)
best_params = search.best_params_
```

Classification Report Comparison

Before Tuning

	precision	recall	f1-score	support
0	0.76	0.54	0.63	198
1	0.94	0.98	0.96	1402
accuracy			0.92	1600
macro avg	0.85	0.76	0.79	1600
weighted avg	0.92	0.92	0.92	1600

After Tuning

=== Threshold = 0.45 ===				
	precision	recall	f1-score	support
Class 0	0.83	0.54	0.65	198
Class 1	0.94	0.99	0.96	1402
accuracy			0.93	1600
macro avg	0.89	0.76	0.81	1600
weighted avg	0.92	0.93	0.92	1600

MODEL BUILDING

Deep Neural Network

We also wanted to see how DNN handles this dataset but the result were not as good as our previous model

```
model = Sequential([
    Dense(256, activation='relu', input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.4),

    Dense(128, activation='relu'),
    BatchNormalization(),
    Dropout(0.3),

    Dense(64, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),

    Dense(1, activation='sigmoid') # Binary Classification
])
```

```
model.compile(
    optimizer=Adam(learning_rate=0.0005),
    loss='binary_crossentropy',
    metrics=['AUC']
)
```

```
history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=64,
    validation_split=0.2,
    class_weight=class_weight_dict,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
```

Classification Report Comparison

DNN Weighted

	precision	recall	f1-score	support
Class 0	0.46	0.51	0.49	198
Class 1	0.93	0.92	0.92	1402
accuracy			0.87	1600
macro avg	0.70	0.71	0.70	1600
weighted avg	0.87	0.87	0.87	1600

XGBoost Weighted and Tuned

=== Threshold = 0.45 ===				
	precision	recall	f1-score	support
Class 0	0.83	0.54	0.65	198
Class 1	0.94	0.99	0.96	1402
accuracy			0.93	1600
macro avg	0.89	0.76	0.81	1600
weighted avg	0.92	0.93	0.92	1600

RESULT

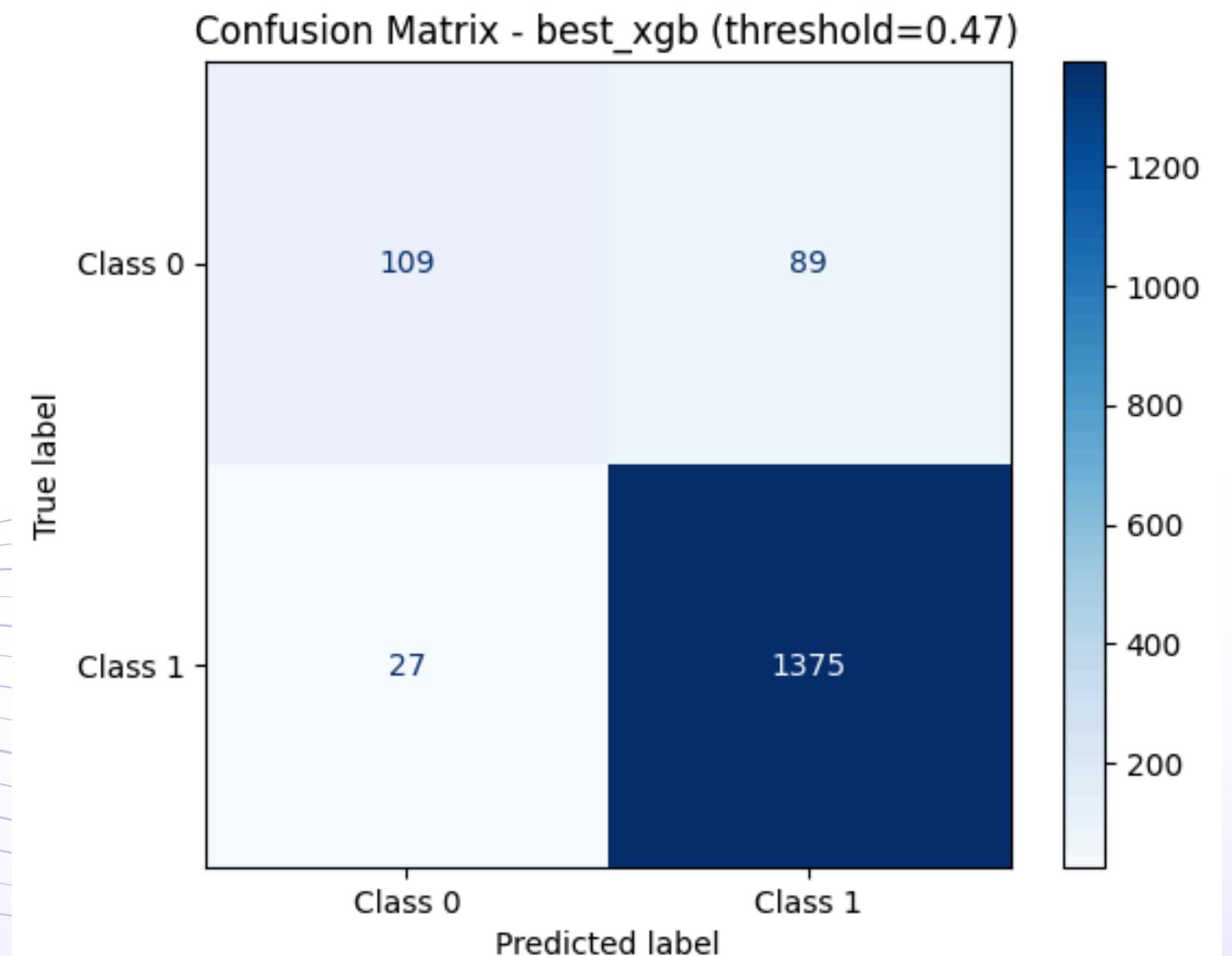
Final Model

We chose the optimized XGBoost model because it gives us high precision and recall on Class 1 (the customers who actually convert), while maintaining as acceptable performance as possible on Class 0.

In marketing terms: We can confidently spend money on high-potential customers and avoid wasting budget on unlikely ones.

```
=== Threshold = 0.45 ===
```

	precision	recall	f1-score	support
Class 0	0.83	0.54	0.65	198
Class 1	0.94	0.99	0.96	1402
accuracy			0.93	1600
macro avg	0.89	0.76	0.81	1600
weighted avg	0.92	0.93	0.92	1600



REAL WORLD IMPACT

KPI 1: Maximum Market Coverage (Class 1 Recall: 99%)

We have effectively eliminated revenue leakage. Our model is tuned to ensure that almost no potential converter is left behind, giving the marketing team total confidence in their reach.

KPI 2: Reliable Resource Filtering (Class 0 Precision: 83%)

While we are aggressive in our pursuit of sales, we aren't flying blind. When our model flags a customer as 'unlikely to buy,' it is correct 83% of the time. This allows us to protect 83% of the budget we would have otherwise spent on 'dead-end' leads."

The "Fairness" Result (Macro F1: 0.81)

By achieving a 0.81 Macro F1, we've proven that the model isn't just 'guessing' based on the majority. It has learned the underlying patterns of both types of customers, providing a stable 93% overall accuracy for the business.

FEATURE IMPORTANCE

What Actually Drives Conversion

Past behaviour & campaign focus: Previous purchases and conversion-targeted campaigns are much more predictive than any other feature.

- Prioritise retargeting existing customers and ensure campaign messaging is conversion-focused, not just awareness.

Engagement quality: Email clicks, time on site, and CTR cluster tightly as the second strongest tier, how a customer behaves after arrival matters more than which channel brought them.

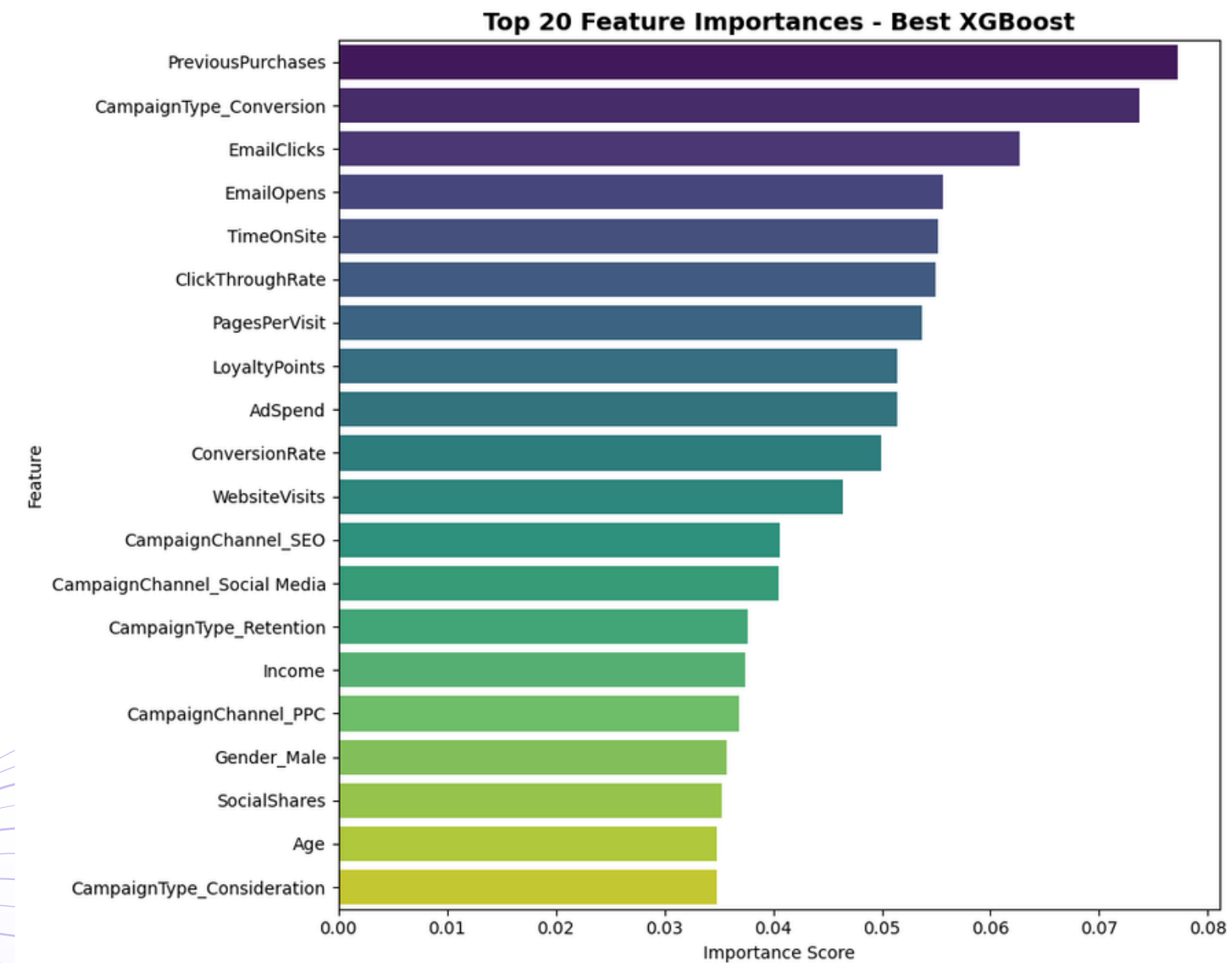
- Invest in content and UX quality over channel diversification.

Loyalty and spend: Loyalty points and ad spend have meaningful but secondary influence.

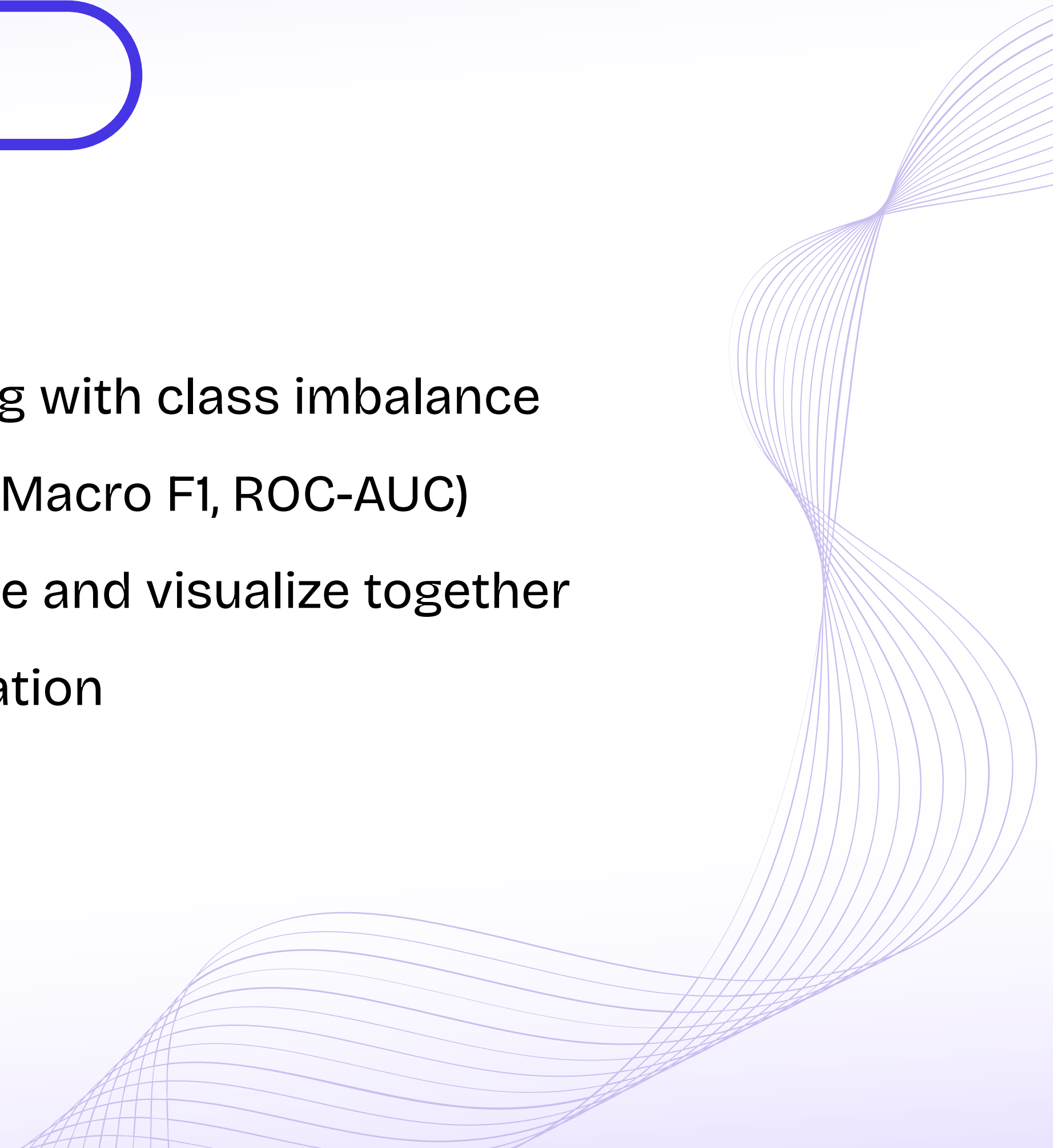
- Loyalty programmes are worth maintaining, but won't compensate for weak engagement or poor campaign targeting.

Demographics barely matter Age, gender, and income sit at the bottom, conversion happens across all segments.

- Avoid over-segmenting campaigns by demographics; behavioural targeting will outperform it.



CHALLENGES & LESSONS

- Learning so many new concepts
 - Especially in relations to dealing with class imbalance
 - Choosing Metric (Precision, Recall, Macro F1, ROC-AUC)
 - Keeping track of results to compare and visualize together
 - Flow organization and experimentation
- 
- A decorative graphic consisting of multiple thin, light blue wavy lines that flow from the bottom left towards the top right, creating a sense of movement and elegance.

REFERENCE

Dataset Source: Predict Conversion in Digital Marketing Dataset

<https://www.kaggle.com/datasets/rabieelkharoua/predict-conversion-in-digital-marketing-dataset>

Nyembe, A. (2026, April 9). 6 Machine learning models using conversion data compared. Madgicx. <https://madgicx.com/blog/machine-learning-models-using-conversion-data>

George, M. (2025, July 4). Precision vs. Recall in Cross-Sell Models - growth-onomics. Growth-onomics. <https://growth-onomics.com/precision-vs-recall-in-cross-sell-models/#:~:text=Deciding%20whether%20to%20prioritize%20precision,recall%20in%20cross%2Dsell%20models?>

GeeksforGeeks. (2026, March 19). XGBoost. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/xgboost/>

SOURCE CODE

<https://colab.research.google.com/drive/15M3q7k0SmXniXlwazOoeKvP8f2Wc2MFN?usp=sharing>

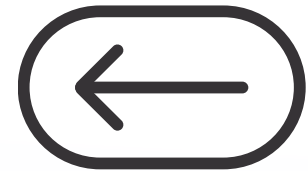
Responsibility

We both researched and experiment with different models and tuning techniques before settling in one flow and final conclusion

Pissoth: EDA, Data Cleaning and Preprocessing, Resampling methods, initial models evaluation with precisions, recalls and weighted macro avg F1

Sovitou: Weights balancing, hypertuning XGBoost

THANK YOU



+ + + +

